

L01 · Εισαγωγικά

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Τι είναι αλγόριθμος, πώς μετράμε την «καλή» επίδοσή του, και ποια θα είναι η ροή του εξαμήνου. Στιγμιότυπο, διάσταση, στοιχειώδης πράξη, τρεις τύποι πολυπλοκότητας.

1 Καλώς ήρθες

Αυτή είναι η αφετηρία για όλο το μάθημα. Στόχος της διάλεξης: μέχρι το τέλος, να ξέρεις **τι ακριβώς εννοούμε όταν λέμε «αλγόριθμος»**, με ποιο εργαλείο τον συγκρίνουμε με άλλους, και ποια διαδρομή θα ακολουθήσουμε από εδώ μέχρι την εξεταστική.

Διαίσθηση

Πρακτικά: ένας αλγόριθμος είναι μια συνταγή. Μια συνταγή μαγειρικής σου λέει «βάλε 2 αυγά, ανακάτεψε για 30 δευτερόλεπτα» — βήματα ξεκάθαρα, σε σειρά, και μετά από πεπερασμένη ώρα έχεις φαγητό. Ένας αλγόριθμος κάνει το ίδιο για ένα **υπολογιστικό πρόβλημα**.

2 Από πού έρχεται η λέξη «αλγόριθμος»

Από τον **Al-Khwarizmi** — Πέρση μαθηματικό του 9ου αιώνα. Έγραψε ένα βιβλίο για μεθοδικές διαδικασίες λύσης εξισώσεων· όταν το έργο του μεταφράστηκε στα Λατινικά, το όνομά του παραφθάρηκε σε *Algoritmi*, και η λέξη επιβίωσε.

3 Γιατί χάνουμε χρόνο σε ένα μάθημα Αλγορίθμων;

Εύλογο ερώτημα — ειδικά τώρα που υπάρχουν LLMs που λύνουν ασκήσεις. Δύο απαντήσεις:

1. **Τα παραδείγματα είναι παντού.** Κοινωνικά δίκτυα (ποιους φίλους να σου προτείνει το Instagram), επικοινωνία drones (σε ποιο συντομότερο μονοπάτι να πάει το επόμενο), εύρεση διαδρομών (Google Maps), συναλλαγές (κρυπτογραφία). Κάτω από όλα αυτά **κάποιος αλγόριθμος** τρέχει, και η ποιότητά του καθορίζει αν η εφαρμογή δουλεύει σε εκατομμύρια χρήστες ή κολλάει στα 100.
2. **Τα LLMs σου λένε τι, όχι γιατί.** Όταν στην εξέταση δεις μια άσκηση τύπου «σχεδίασε αλγόριθμο που...» θα πρέπει να αναγνωρίσεις **ποιο σχήμα** εφαρμόζεται, να αποδείξεις ότι δουλεύει, και να αναλύσεις την πολυπλοκότητα. Αυτό το μάθημα είναι ακριβώς αυτή η εξάσκηση.

4 Οι τρεις στόχοι του εξαμήνου

Όλα όσα κάνουμε καταλήγουν σε ένα από αυτά τα τρία:

1. **Αποτίμηση** — δοθέντος ενός αλγορίθμου, πόσο χρόνο και μνήμη χρειάζεται;
2. **Σύγκριση** — δοθέντων δύο αλγορίθμων που λύνουν το ίδιο πρόβλημα, ποιος είναι καλύτερος και υπό ποιες προϋποθέσεις;
3. **Σχεδίαση** — δοθέντος ενός προβλήματος που δεν έχουμε ξαναδεί, πώς στήνουμε έναν καλό αλγόριθμο για αυτό;

5 Δύο σημαντικές ερωτήσεις πριν προχωρήσουμε

5.1 Είναι όλα τα προβλήματα επιλύσιμα από έναν αλγόριθμο;

Όχι. Υπάρχουν προβλήματα που είναι αποδεδειγμένα **μη επιλύσιμα** από οποιονδήποτε αλγόριθμο (το διάσημο παράδειγμα: το *halting problem* — «δοθέντος ενός προγράμματος, θα τερματίσει;»). Στο μάθημα δεν θα μπούμε σε υπολογιστική μη-αποφασισσιμότητα, αλλά αξίζει να ξέρεις ότι υπάρχει αυτό το όριο.

5.2 Υπάρχει για όλα τα επιλύσιμα προβλήματα ένας «καλός» αλγόριθμος;

Αυτή είναι η ερώτηση που γέννησε ολόκληρο τον τομέα της **πολυπλοκότητας**. Υπάρχουν προβλήματα **επιλύσιμα μεν**, αλλά κανείς δεν ξέρει αν υπάρχει «γρήγορος» (=πολυωνυμικός) αλγόριθμος για αυτά. Το αν $P = NP$ είναι ένα από τα 7 «Millennium problems» με 1 εκ. δολάρια έπαθλο.

Κλειδί

Πρόγραμμα και Αλγόριθμος. Κάθε πρόγραμμα υλοποιεί κάποιον αλγόριθμο. Το αντίστροφο όχι πάντα ευθέως: ένας αλγόριθμος μπορεί να περιγραφεί σε ψευδογλώσσα, αλλά για να τρέξει χρειάζεται προγραμματισμό σε συγκεκριμένη γλώσσα. **Για το μάθημα τους θεωρούμε ισοδύναμους:** Αλγόριθμος $\leftrightarrow$ Πρόγραμμα.

6 Παράδειγμα: η ψευδογλώσσα μας

Πριν μιλήσουμε για πολυπλοκότητα, ας δούμε πώς γράφουμε έναν αλγόριθμο **σε ψευδογλώσσα**. Εδώ είναι η δυαδική αναζήτηση σε ταξινομημένο πίνακα (π.χ. τηλεφωνικός κατάλογος):

Αλγόριθμος ΔυαδικήΑναζήτηση(Π , x):

Επανέλαβε:

 Σύγκρινε το x με το μεσαίο στοιχείο του Π

 Αν είναι μικρότερο: συνέχισε στο πρώτο ήμισυ

 Αλλιώς: συνέχισε στο δεύτερο ήμισυ

Έως (x == μεσαίο στοιχείο) ή (ο πίνακας είναι κενός)

Δες πώς δουλεύει σε ένα μικρό παράδειγμα — αναζήτηση του 23 σε ταξινομημένο πίνακα 8 στοιχείων, [5, 11, 14, 17, 23, 28, 33, 41]:

- **Βήμα 1:** μεσαίο στοιχείο = 17. Επειδή $23 > 17$, η αναζήτηση συνεχίζει στο δεξί μισό [23, 28, 33, 41].
- **Βήμα 2:** μεσαίο στοιχείο = 33. Επειδή $23 < 33$, η αναζήτηση συνεχίζει στο αριστερό μισό [23, 28].
- **Βήμα 3:** μεσαίο στοιχείο = 23. Ισότητα — το στοιχείο βρέθηκε.

Σε πίνακα 8 στοιχείων χρειάστηκαν **3 συγκρίσεις**. Σε πίνακα 1.000.000 στοιχείων θα χρειαστούν περίπου **20**. Αυτή η συμπεριφορά «κάθε βήμα κόβει στη μέση» είναι ο πυρήνας του τι μελετάμε στο επόμενο μάθημα, την ασυμπτωτική ανάλυση, και επιστρέφει αργότερα στο Divide & Conquer.

7 Πότε ένα πρόγραμμα είναι «χρήσιμο»;

Σύμφωνα με τη διάλεξη:

Ένα πρόγραμμα είναι χρήσιμο όταν, **σε λογικό χρόνο** και **σε λογικό χώρο μνήμης**, εξάγει το **σωστό αποτέλεσμα**.

Αυτή η μία πρόταση κρύβει τρεις απαιτήσεις:

Απαίτηση	Τι μετράμε
Ορθότητα	Το αποτέλεσμα είναι αυτό που υποσχέθηκε ο αλγόριθμος;
Χρόνος	Πόσες στοιχειώδεις πράξεις χρειάζονται;
Μνήμη	Πόσο επιπλέον χώρο καταναλώνει;

Οι δύο τελευταίες μαζί αποτελούν την **πολυπλοκότητα** του αλγορίθμου.

8 Πρόβλημα, στιγμιότυπο, διάσταση — τρία διαφορετικά πράγματα

Είναι εύκολο να τα μπερδέψεις, αλλά κάθε άσκηση εξετάσεων θα σου πει ξεκάθαρα και τα τρία. Πάμε με δύο παραδείγματα.

Παράδειγμα

Πρόβλημα 1 — Ταξινόμηση

- *Είσοδος:* n ακέραιοι $a_1, a_2, \dots, a_n$.
- *Ζητούμενο:* να ταξινομήσουμε τους ακέραιους σε αύξουσα τάξη.

Στιγμιότυπο ενός τέτοιου προβλήματος είναι π.χ. $\{8, 3, 5, 1, 9\}$ — ένας συγκεκριμένος πίνακας 5 ακεραίων.

Διάσταση του στιγμιότυπου είναι το $n = 5$ — δηλαδή ο αριθμός των στοιχείων.

Παράδειγμα

Πρόβλημα 2 — Knapsack (σακίδιο)

- *Είσοδος:* n αντικείμενα με αξίες $c[i]$ και όγκους $w[i]$, και ένα σακίδιο χωρητικότητας b .
- *Ζητούμενο:* να επιλέξουμε σύνολο αντικειμένων με τη μεγαλύτερη αξία που να χωράνε στο σακίδιο.

Στιγμιότυπο: ένα συγκεκριμένο σύνολο αντικειμένων με συγκεκριμένες αξίες/όγκους και συγκεκριμένη χωρητικότητα.

Διάσταση: το n — δηλαδή πάλι ο αριθμός των αντικειμένων.

Κλειδί

Διάκριση — *Πρόβλημα* = γενική περιγραφή. *Στιγμιότυπο* = συγκεκριμένα δεδομένα ενός παραδείγματος. *Διάσταση* = αριθμητική παράμετρος που μετράει το «μέγεθος» του στιγμιότυπου. Πάντα η πολυπλοκότητα εκφράζεται ως **συνάρτηση της διάστασης**.

9 Πώς μετράμε την πολυπλοκότητα

- **Μονάδα μέτρησης:** η στοιχειώδης (ουσιώδης) πράξη — μια ενέργεια που υποθέτουμε ότι γίνεται σε σταθερό χρόνο. Παραδείγματα: μία πρόσθεση, μία σύγκριση, μία εκχώρηση σε μεταβλητή.
- **Συνάρτηση της διάστασης:** μετράμε πόσες στοιχειώδεις πράξεις γίνονται ως συνάρτηση του n .

Η συνάρτηση αυτή είναι ο **χρόνος εκτέλεσης** του αλγορίθμου, και θα μάθουμε στο επόμενο μάθημα πώς τη συγκρίνουμε με άλλες χρησιμοποιώντας τα O , Θ , Ω .

10 Τρεις τύποι πολυπλοκότητας

Όταν λέμε «πόσο χρόνο κάνει ο αλγόριθμος;», για ποιο στιγμιότυπο μιλάμε; Διαφορετικά στιγμιότυπα της ίδιας διάστασης μπορεί να απαιτούν διαφορετικό χρόνο. Διακρίνουμε τρεις τύπους:

- **Βέλτιστη περίπτωση** ($C_{\beta\pi}$) — το «πιο εύκολο» στιγμιότυπο.
- **Χείριστη περίπτωση** ($C_{\chi\pi}$) — το «πιο δύσκολο» στιγμιότυπο.
- **Μέση περίπτωση** ($C_{\mu\pi}$) — σταθμισμένος μέσος πάνω σε όλα τα στιγμιότυπα.

Πιο τυπικά, αν D_n είναι το σύνολο όλων των στιγμιότυπων διάστασης n :

$$C_{\beta\pi}(n) = \min_{d \in D_n} \text{κόστος}(d) \quad (\text{βέλτιστη περίπτωση})$$

$$C_{\chi\pi}(n) = \max_{d \in D_n} \text{κόστος}(d) \quad (\text{χείριστη περίπτωση})$$

$$C_{\mu\pi}(n) = \sum_{d \in D_n} p(d) \cdot \text{κόστος}(d) \quad (\text{μέση περίπτωση})$$

όπου $p(d)$ η πιθανότητα να εμφανιστεί το στιγμιότυπο d ως είσοδος.

Προσοχή

Σχεδόν πάντα στις εξετάσεις χρησιμοποιούμε χείριστη περίπτωση. Είναι η εγγύηση — «ο αλγόριθμος δεν θα πάρει ποτέ παραπάνω από αυτό». Η μέση περίπτωση χρειάζεται υποθέσεις για την κατανομή της εισόδου, και αυτό σπάνια είναι ξεκάθαρο.

11 Επεξεργασμένο παράδειγμα: γραμμική αναζήτηση

Παράδειγμα

Πρόβλημα. Έχουμε πίνακα $S = (a_1, a_2, \dots, a_n)$ και ζητάμε ένα στοιχείο x . Επιστρέφουμε τη θέση του x στον πίνακα, αν υπάρχει.

Αλγόριθμος.

```
i <- 1
Όσο (i != n+1) και (a_i != x):
    i <- i + 1
Εάν i > n: εκτύπωσε("όχι")
Αλλιώς: εκτύπωσε("στοιχείο x στη θέση i")
```

Ανάλυση (βασική στοιχειώδης πράξη: η σύγκριση $a_i \neq x$):

- *Βέλτιστη περίπτωση:* το x βρίσκεται στη **θέση 1**. Μία σύγκριση. $C_{\beta\pi}(n) = 1$.
- *Χείριστη περίπτωση:* το x είτε δεν υπάρχει, είτε βρίσκεται στη **θέση n**. Πρέπει να γίνουν n συγκρίσεις. $C_{\chi\pi}(n) = n$.
- *Μέση περίπτωση:* αν υποθέσουμε ομοιόμορφη κατανομή του x στις θέσεις 1 μέχρι n , ο αναμενόμενος αριθμός συγκρίσεων είναι $\frac{1+2+\dots+n}{n} = \frac{n+1}{2}$, δηλαδή και πάλι γραμμικός στο n αλλά με σταθερά $1/2$.

Συμπέρασμα. Η γραμμική αναζήτηση έχει χείριστη πολυπλοκότητα **γραμμική** στο n . Η δυαδική αναζήτηση που είδαμε νωρίτερα έχει **λογαριθμική** χείριστη πολυπλοκότητα — γι' αυτό σε μεγάλα δεδομένα η δεύτερη είναι ασύγκριτα ταχύτερη. Στο επόμενο μάθημα θα μάθουμε να εκφράζουμε αυτή τη διαφορά τυπικά: $O(n)$ vs $O(\log n)$.

12 Δεύτερο παράδειγμα: αριθμητικό γινόμενο

Παράδειγμα

Πρόβλημα. Δίνονται δύο διανύσματα $a = (a_1, \dots, a_n)$ και $b = (b_1, \dots, b_n)$. Υπολόγισε το **αριθμητικό γινόμενο** $sp = \sum_{k=1}^n a_k \cdot b_k$.

Αλγόριθμος.

```
sp <- 0
Για k από 1 έως n:
    sp <- sp + a_k * b_k
Αριθμητικό γινόμενο = sp
```

Ανάλυση. Ο βρόχος εκτελείται ακριβώς n φορές, ανεξάρτητα από την είσοδο. Κάθε επανάληψη κάνει σταθερό αριθμό πράξεων (1 πολλαπλασιασμό + 1 πρόσθεση + 1 ενημέρωση δείκτη). Άρα:

$$C_{\beta\pi}(n) = C_{\chi\pi}(n) = C_{\mu\pi}(n) = \Theta(n)$$

Παρατήρηση. Όταν δεν υπάρχει «εύκολη» ή «δύσκολη» είσοδος — όταν ο αλγόριθμος κάνει την ίδια ακριβώς δουλειά για όλα τα στιγμιότυπα της ίδιας διάστασης — οι τρεις τύποι πολυπλοκότητας **συμπίπτουν**.

13 Το πλάνο του εξαμήνου

Τέσσερις τεχνικές σχεδιασμού αλγορίθμων, μία θεματική ενότητα γραφημάτων, και μία ενότητα δομών δεδομένων — όλα κάθονται πάνω στα θεμέλια που χτίζουμε στα δύο πρώτα μαθήματα:

- **L01–L02:** Εισαγωγικά & Ασυμπτωτική ανάλυση (τα θεμέλια).
- **L03–L05:** Divide & Conquer.
- **L06–L09:** Γραφήματα.
- **L10:** Δομές Δεδομένων.
- **L11–L13:** Άπληστοι αλγόριθμοι (greedy).
- **L14–L17:** Δυναμικός προγραμματισμός.

Η ασυμπτωτική ανάλυση είναι **παντού**: σε κάθε ανάλυση χρόνου που θα κάνουμε, σε κάθε σύγκριση αλγορίθμων, σε κάθε απόδειξη βελτιστότητας. Γι' αυτό την παίρνουμε πρώτη.

14 Βαθμολογία και πρακτικά

- Δύο βαθμοί: **Πρόοδος (Π)** και **Γραπτό (Γ)**.
- **Τελικός βαθμός** = $\max\{\Gamma, 0.3 \cdot \Pi + 0.7 \cdot \Gamma\}$ — δηλαδή η πρόοδος βοηθάει, δεν βλάπτει: αν τα πας καλά στην πρόοδο και χάλια στο γραπτό, σε «τραβάει» πάνω· αν τα πας καλύτερα στο γραπτό, αγνοείται η πρόοδος.
- Όριο 40% στο γραπτό για να μετρήσει η πρόοδος.

Σημείωση

Συνιστώμενα προαπαιτούμενα: Διακριτά Μαθηματικά, Δομές Δεδομένων & Τεχνικές Προγραμματισμού. Αν δεν τα έχεις δει — οι έννοιες που θα χρειαστούμε χτίζονται μέσα στο μάθημα, αλλά θα πρέπει να επενδύσεις λίγο περισσότερο σε άσκηση.

15 Βιβλιογραφία

- **Cormen, Leiserson, Rivest, Stein** — *Introduction to Algorithms* (CLRS). Η «βίβλος» των αλγορίθμων.
- **Kleinberg & Tardos** — *Algorithm Design*. Πιο φιλική στην απόδειξη ορθότητας, εξαιρετική στο D&C και το greedy.
- **Dasgupta, Papadimitriou, Vazirani** — *Algorithms*. Συνοπτικότερο, διαθέσιμο ελεύθερα σε pdf.
- Συμπληρωματικά: σημειώσεις και διαφάνειες **B. Ζησιμόπουλου**.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. Ένας **αλγόριθμος** είναι μια καλά ορισμένη ακολουθία βημάτων που μετασχηματίζει είσοδο σε έξοδο.
2. Ένα **πρόβλημα** ορίζει είσοδο/έξοδο. Ένα **στιγμιότυπο** είναι μια συγκεκριμένη είσοδος. Η

διάσταση είναι η αριθμητική παράμετρος που μετράει το μέγεθος του στιγμιοτύπου — συνήθως το n .

3. Η **πολυπλοκότητα** είναι ο αριθμός στοιχειωδών πράξεων ως συνάρτηση της διάστασης.
4. **Τρεις τύποι**: βέλτιστη, χείριστη, μέση περίπτωση. Σχεδόν πάντα χρησιμοποιούμε **χείριστη**.
5. Στο μάθημα: 4 τεχνικές σχεδιασμού (D&C, Greedy, DP) + Γραφήματα + Δομές Δεδομένων, με την ασυμπτωτική ανάλυση να ενώνει τα πάντα.

Σημειώσεις από το *Algorithms Class Hub* — μελετητικός οδηγός για το μάθημα «Αλγόριθμοι και Πολυπλοκότητα (Κ17)», ΕΚΠΑ.